AMR PROJECT GUIDE

Explanation, MATLAB files, graphs and live demo cheat sheet

Design and Simulation of Industry Standard AMRs (AgileX Scout Mini) with Motor, Torque and Slip Model

Sailakshmi B T (2024511019) | RA23502 Autonomous Mobile Robot

Part 1 Project explanation, MATLAB files and every graph explained

Part 2 Live demo cheat sheet: the command for every figure

MATLAB R2025b | Simulink, Simscape, Simscape Multibody

Part 1: Project Explanation, MATLAB Files and Graphs

Design and Simulation of Industry Standard AMRs (AgileX Scout Mini) with Motor, Torque and Slip Model

Sailakshmi B T (2024511019) | RA23502 Autonomous Mobile Robot

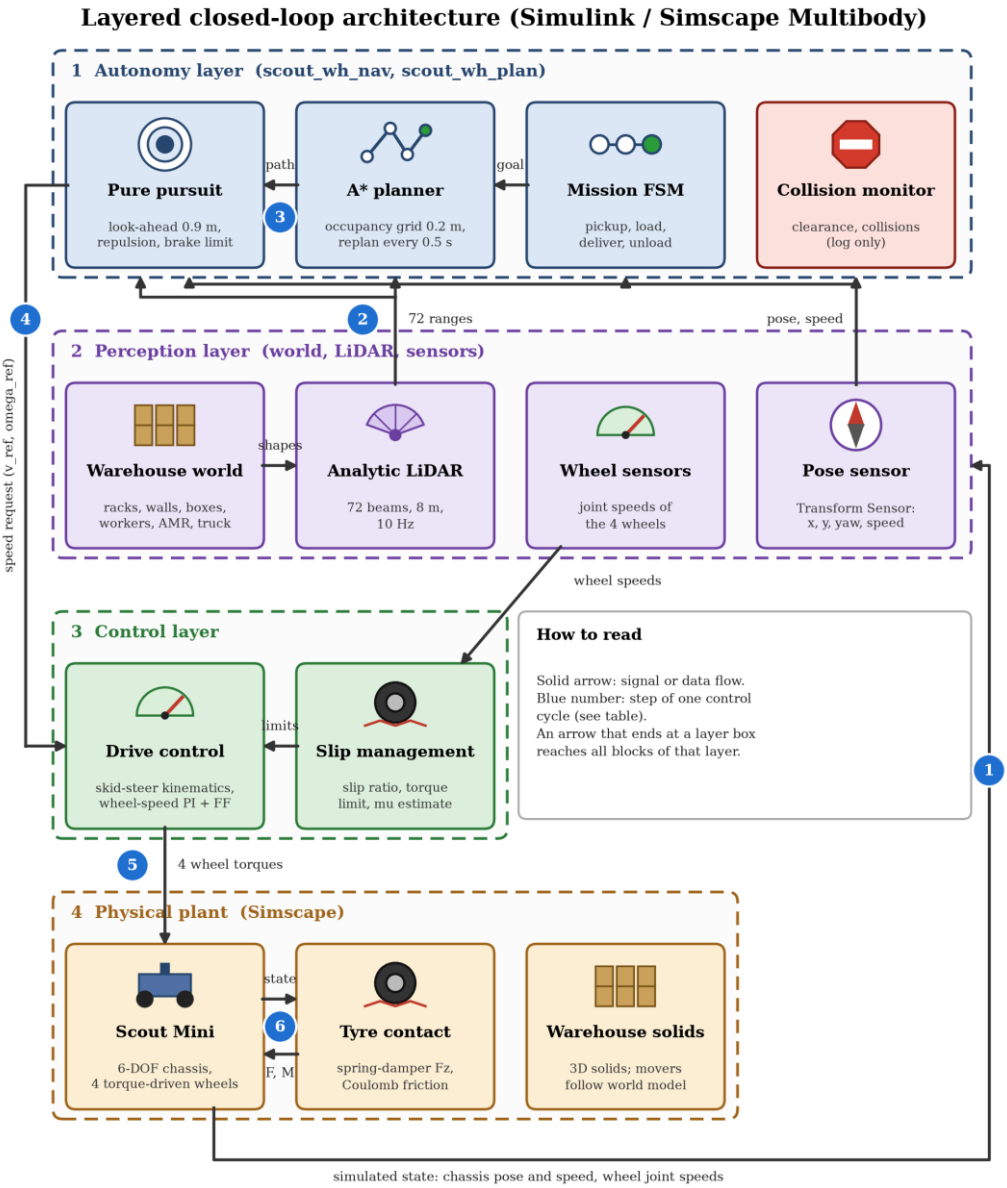
This document explains the project report in simple words and lists every MATLAB file, what it does and what output it gives. Use it to understand the work and to prepare for questions. The folder is D:\academic_shit\SEM5-III\AMR ROS\simulink_amr unless another path is given. The numbers are those of the latest run, with the motor and battery model.

0. Explain it simply (start here)

The two-minute story for your professor:

- **The idea.** A warehouse robot has to carry goods from a pickup point to a delivery point without a driver and without hitting anything. Testing that on a real floor is slow and risky, so I built the robot and the warehouse inside the computer and let the robot do the job by itself.
- **The robot.** It is the AgileX Scout Mini, a small four-wheel robot. I drew it in SolidWorks, exported it as a URDF file (a text file that lists the robot's parts, joints and masses) plus 3D shape files, and loaded it into MATLAB Simulink / Simscape Multibody.
- **The physics.** Simscape Multibody is a physics engine: the robot has real mass, the four wheels get a motor torque, and the floor pushes back with friction. I wrote the wheel-floor friction model myself, with dry concrete, oil and water patches, so wheels can really slip.
- **The warehouse.** Racks with goods, walls, a pickup and a delivery station, floor boxes, two workers, a second robot and a pallet truck that blocks an aisle.
- **The brain.** Four steps repeated all the time: (1) LiDAR looks around, (2) the robot updates its map, (3) A* finds the shortest free path and plans again when something blocks it, (4) pure pursuit steers along the path while a safety rule slows or stops the robot near obstacles.
- **The wheels.** A slip controller watches each wheel. If a wheel spins faster than the ground moves, it reduces the torque, so the robot keeps its grip.
- **The motors and the battery.** Each wheel has a hub motor (24 V, 5 N·m rated, 15 N·m peak). The torque it can give falls as the wheel turns faster, and the simulation obeys this limit. From the logged torques and wheel speeds the motor current, the power and the battery voltage are calculated: 9.2 A peak current, 1.06 Wh used, battery never below 24.4 V.
- **The result.** The robot finished the job in 51.5 s, drove 32.5 m, changed its route 5 times and had 0 collisions (closest it came to anything: 0.12 m). With slip control the worst slip was 0.46, without it 0.63.

The architecture in plain words (Fig. 21 of the report). Think of four layers that form a loop:



Layer	In plain words	Where it is
4 Physical plant	The 'real world' inside the computer: the robot, its wheels, the floor friction and the warehouse. It obeys physics.	scout_amr_sim.slx (Simscape Multibody), scout_amr_tires.m
2 Perception	The robot's eyes: where am I and how fast am I going, what is around me. The LiDAR sends 72 rays and measures the distance to the first thing each ray hits.	Transform Sensor blocks, scout_wh_world.m, scout_wh_lidar.m
1 Autonomy	The decision maker: mission steps (pickup, load, deliver, unload), the map, the A* route, and the steering with obstacle avoidance. Its output is 'drive at this speed and turn at this rate'.	scout_wh_nav.m, scout_wh_plan.m
3 Control	The muscle: converts 'speed and turn rate' into a target speed for each wheel, then into a motor torque, and limits the torque when a wheel slips.	scout_amr_controller.m

The loop: the plant produces the state, perception reads it, autonomy decides, control sends wheel torques, the tyre model turns them into forces, and the plant moves and produces a new state. This repeats every 5 ms. The collision monitor only watches and records; the robot never uses it to steer.

Words you may need:

Term	Simple meaning
AMR	Autonomous mobile robot: it finds its own way, unlike a robot that follows a line on the floor.
URDF / STL	URDF = text description of the robot's parts and joints; STL = the 3D shape of each part.
Skid steer	Left and right wheels turn at different speeds to steer, like a tank. There is no steering wheel.
LiDAR	A spinning distance sensor: it measures how far the nearest object is in each direction.
Occupancy grid	A map made of small squares (20 cm); each square is free or occupied. Obstacles are enlarged by 0.48 m so the robot keeps a safe distance.
A*	A search method that finds the shortest path through the free squares, guided by the straight-line distance to the goal.
Pure pursuit	The robot always steers towards a point 0.9 m ahead on its path.
Slip ratio	How much faster a wheel turns than the ground moves under it. 0 = perfect grip, large = the wheel is spinning.
Traction control	Reduces the wheel torque when slip is too high, so the wheel grips again.
PI controller	A simple feedback rule: torque = gain x speed error + accumulated error. It makes each wheel reach its target speed.

If asked what is NOT real (say it first): it is simulation only; the pose comes straight from the simulator; the LiDAR and the collision check are computed from the geometry; the workers and the truck follow scripts; nothing was tested on the real robot.

1. The project in one page

What was built. A virtual warehouse robot. The robot is your own SolidWorks model of the AgileX Scout Mini (a four-wheel skid-steer robot). It is placed in a 3D warehouse inside MATLAB Simulink / Simscape Multibody, where real physics (mass, wheel torque, tyre friction) moves it. The robot drives by itself from a start dock to a pickup station, waits to load, drives to a delivery station and waits to unload, while avoiding racks, boxes, a worker, a second robot and a pallet truck.

The chain of work, in order:

- **SolidWorks CAD model** of the robot (your files Assem2, chassis, chassis_top, W, up).
- **Export to URDF + STL meshes**: a text description of the robot (links, joints, masses) plus the 3D shapes.
- **Import into MATLAB**: importrobot (for viewing) and smimport (creates the Simscape Multibody / Simulink model).
- **Make it a physical robot**: free-moving chassis, torque-driven wheels, a tyre-ground friction model written by us.
- **Build the warehouse**: racks with goods, walls, stations, floor boxes, oil/water patches, and four moving obstacles.
- **Add autonomy**: LiDAR, occupancy grid, A* path planning, re-planning, path following and a mission state machine.
- **Add slip management**: detect wheel slip and limit torque so the wheels do not spin or lock.
- **Run, log, plot, report**: one script runs everything and draws all graphs.

Why it matters. Testing a warehouse robot in simulation is cheaper and safer than on a real floor. The project shows the full path from a CAD model to a robot that senses, plans, moves and handles slippery floors.

2. The report explained section by section

Section	What it tells	Key point to remember
Abstract	One-paragraph summary of the whole project and its results.	Mission 51.5 s, 5 route corrections, 0 collisions, minimum clearance 0.12 m.
1 Introduction	What an AMR is, why simulate, the objectives and the scope.	Simulation only. Real hardware was not tested.
2 Robot Specifications	Numbers of the model: mass, wheel size, track, motor limit, tyre values.	Values come from your CAD/URDF; the motor and battery data are assumed (the real robot was not measured).
3 Sensors	Every sensor, its configuration, why it is used and whether it is really simulated.	Only the LiDAR (analytic) and the body-state / wheel-speed sensors are used in Simulink. IMU and camera are mounted but give no signals.
4 SolidWorks Design	Your CAD files and screenshots.	SolidWorks files cannot be opened in MATLAB, so STL meshes carry the shape.
5 Conversion to URDF	What URDF is and the values read from it.	Only change made: package:// mesh paths replaced by meshes/.
6 Importing in MATLAB and Simulink	importrobot and smimport.	smimport is slow (minutes), so the result is cached in scout_import_raw.slx.
7 Building the Simscape Multibody Model	Steps that turn the import into a dynamic robot, the tyre model, the subsystem structure, the top-level screenshot and the controller diagram.	6-DOF chassis, torque-driven wheels, spring-damper normal force + smoothed Coulomb friction.
8 Creating the Warehouse	Layout table, moving obstacles, planning map and the Multibody Explorer screenshots.	Racks and walls are known to the planner; boxes, workers, AMR and truck are found only by LiDAR.
9 Autonomous Navigation	One full mission with the state machine, the result figures and the limits.	All six demo scenarios happen in one mission.
10 Slip and Traction Management	How slip is detected and limited; ON versus OFF comparison.	Peak slip 0.46 (ON) vs 0.63 (OFF); ON is about 6.0 s slower. Both finished without collision.
11 Algorithms	Table of every algorithm, file and parameters.	Reactive avoidance: moving obstacles are not predicted.
12 System Architecture	Layered closed-loop diagram and the flow of one 5 ms control step.	Four layers in a loop: Plant, Perception, Autonomy, Control.
13 Additional MATLAB Visual Simulation	The script simulation with ISO 3691-4 style safety fields.	Kinematic animation, not a multibody solver.
14 How to Set It Up	Requirements, run commands, parameters, troubleshooting, live graphs (14.8).	Run run_scout_warehouse_simulation or live_graphs.
15 Mathematical Formulation / 16 Verification	Equations of the implementation and the verification checklist.	Every result is checked against a limit (collisions, torque, slip, current).
17 Motor, Torque and Battery Calculations	Motor model, torque for each driving case, grip limit, battery, comparison of hand calculation and simulation.	Hand torque agrees with the simulation (cruise 1.27 N·m simulated); peak current 9.2 A against 23.4 A allowed.
18 Limitations and Future Enhancements	What is not modelled and what to do next.	Analytic LiDAR and collisions, scripted obstacles, true pose, motor current does not act on the dynamics.
19 Conclusion / 20 References	Summary and the references.	Shows the software chain works; not proof for the physical robot.

3. MATLAB files and the output each one gives

The files are grouped by purpose. 'Output' means what you get when the file is run or called.

3.1 Robot description (inputs, not code)

File / folder	What it is	Output / use
`scout_mini.urdf`	URDF of the robot: links, joints, masses, inertias, sensor mounts (copied from the ROS 2 repository; mesh paths changed to meshes/).	Read by importrobot and smimport.
`meshes/ (base_link.STL, w1.STL ... w4.STL)`	3D shapes exported from your SolidWorks model.	Drawn in the 3D scene and in the report renders.
`../scout_mini_sw/ (Assem2.SLDASM, chassis.SLDPRT, chassis_top.SLDPRT, W.SLDPRT, up.SLDPRT)`	Your original SolidWorks files (cloned from GitHub).	Source of the STL meshes. Not read by MATLAB.

3.2 Model and physics

File	What it does	Output
`scout_amr_params.m`	Holds every constant: robot, wheels, tyre contact, friction patches, warehouse layout, LiDAR, planning grid, controller gains and the motor, battery and load data (Kt 0.64 N·m/A, R 0.6 ohm, 24 V, 15 Ah, 5 / 15 N·m rated / peak). Also computes the wheel rotation signs from the URDF.	Returns the struct P used by all other files. Edit this file to change the warehouse or the gains.
`build_scout_amr_model.m`	Imports the URDF with smimport (or reuses scout_import_raw.slx), replaces the weld by a 6-DOF joint, makes wheels torque-driven, adds sensors, tyre forces, the 3D warehouse, the four movers, logging blocks, subsystems and the diagram layout.	Creates scout_amr_sim.slx. Run it only to rebuild the model.
`scout_import_raw.slx`	Raw result of smimport (cache).	Speeds up rebuilds.
`scout_amr_sim.slx`	The Simulink / Simscape Multibody simulation. Top level: World, Mechanism Configuration, Solver and four subsystems (Scout_Mini_Robot with the nested Tyre_Ground_Contact_Model, Warehouse, Moving_Obstacles, Mission_and_Slip_Controller).	When run it logs ctrl_log, tire_log, lidar_log, path_log and mover_log to the workspace and shows the 3D Mechanics Explorer.
`scout_amr_motor_limit.m`	Torque-speed limit of one hub motor: min(15 N·m, Kt (24 V - Ke |w|) / R).	Maximum wheel torque at a given wheel speed; used by the controller, the sizing script and the tests.
`scout_amr_tires.m`	Tyre-ground model for one wheel (called four times): normal spring-damper force, smoothed Coulomb friction, floor friction map.	11 values per wheel: force (3) and torque (3) in the wheel frame, normal load Fz, longitudinal force, lateral force, slip ratio, friction μ.

3.3 Autonomy and control (called every 5 ms from the model)

File	What it does	Output
`scout_amr_controller.m`	The brain inside the Controller block: gets robot pose and speed, calls the navigation, converts commands to wheel targets, estimates slip, applies slip management, runs the wheel-speed PI controller.	tau_j: four wheel torques. lg: log vector of 44 signals (pose, speeds, slip ratios, torques, friction estimate, mission state, clearance, collisions...). mv: 8 mover positions. rng: 72 LiDAR ranges. pth: planned path.
`scout_wh_nav.m`	Perception, planning and mission logic: world update, LiDAR, occupancy grid, A* replanning, pure pursuit, reactive repulsion, stop-before-contact limit, retreat, mission state machine, collision	Speed and yaw-rate reference, mover positions, LiDAR ranges, path and an info vector (state, number of re-routes, clearance, collisions, goal

	monitor.	distance).
scout_amr_drive_sizing.m	Hand calculation of 8 driving cases (cruise, start, brake, dock ramp, pivot, with and without 12 kg load), the grip limits and the motor limits.	Prints and returns the table of force, torque, current, power and % of rated torque (report section 17).
scout_amr_drive_post.m	Motor current, power, battery current, voltage and charge from the logged wheel torques and speeds.	Peak current, mean and peak power, energy, minimum battery voltage, charge at the end.
test_scout_amr_drive.m	Six unit tests of the drive model (stall torque, no-load speed, knee, hand values, grip, battery).	Run with runtests('test_scout_amr_drive'); all six pass.
scout_wh_world.m	Warehouse geometry and moving obstacles at time t (17 rectangles, 2 circles, mover positions). Workers react to the robot; the truck is released at x > 12 m.	Rectangles, circles and the positions that move the 3D bodies.
scout_wh_lidar.m	Casts 72 rays against the rectangles and circles.	72 range values in metres (8 m means no return).
scout_wh_plan.m	8-connected A* on the inflated grid with line-of-sight smoothing.	Path of up to 80 waypoints and their count.

3.4 Running and plotting

File	How to run	Output
run_scout_warehouse_simulation.m	run_scout_warehouse_simulation (3D window) run_scout_warehouse_simulation(false) (headless) run_scout_warehouse_simulation(false,true) (slip ON and OFF) run_scout_warehouse_simulation('replot') (redraw from saved data)	Runs the model, saves results/scout_warehouse_results.mat, draws figures 1 to 5 and the motor and battery figures 7 and 8 (6 with the comparison) as PNG in results/ and prints the summary (mission time, distance, speeds, re-routes, collisions, clearance, slip, energy, motor current, battery).
results/1_warehouse_trajectory.png	(made by the file above)	Map with driven path, planned paths, re-route events and mover trails.
results/2_velocity_wheel_speeds.png		Linear and angular velocity (commanded and measured) and the four wheel speeds.
results/3_perception_safety.png		Closest LiDAR range, ground-truth clearance, route corrections, collisions, mission state.
results/4_slip_management.png		Slip ratio of each wheel, torques after limiting, friction estimate vs true friction, normal loads.
results/5_lidar_scenarios.png		Four snapshots: avoiding the box, worker crossing, aisle blocked re-route, worker in aisle 3.
results/6_slip_on_vs_off.png	(only with compareSlip = true)	Worst-wheel slip ratio and speed with slip management on and off.
results/7_motor_battery.png		Torque-speed limit with the operating points, motor current, battery power, battery voltage and charge.
results/8_drive_design_cases.png		Torque-speed limit of one motor with the 8 hand-calculated driving cases and the grip limits.
results/scout_warehouse_results.mat		Saved logs of both runs (R) and the parameters (P).

live_graphs.m	live_graphs or live_graphs(false)	Runs the mission and updates one figure while the simulation runs: map, speed, slip, clearance, motor current, battery voltage and electrical power.

3.5 Report-making files (folder report/)

File	Output
make_report_figures.m	robot_importrobot.png, robot_render.png (robot meshes with joint axes), the Simulink subsystem pictures (hi-res prints of the Mission controller, used split in two halves as simulink_Mission_2col.png) (in report/figs).
report_numbers.m	numbers.json: every number quoted in the report (ON and OFF run metrics, re-route times).
capture_industry_full.m	industry_01..05, industry_final.png (screenshots) and industry_telemetry.mat: runs simulate_industry_amr_warehouse.m unchanged and grabs its window and telemetry.
make_industry_graphs.m	industry_g1_kinematics.png, industry_g2_safety_sensors.png, industry_g3_path.png redrawn from industry_telemetry.mat.
make_diagrams.py	diag_toolchain.png, diag_urdf_tree.png, diag_robot_subsystem.png, diag_mission_fsm.png, diag_architecture.png, diag_control_step.png (Python, matplotlib; drawn at the printed width so the text stays readable).
content.py + build_report.py	The full report (50 pages) as PDF and .docx (Python; content*.py hold the text, build_report.py the layout).
content_r2.py + build_review2.py	The second-review report (25 pages) as PDF and .docx.
build_guide.py	This guide: Part 1 (explanation, files, graphs) and Part 2 (live demo cheat sheet).

3.6 Earlier visual MATLAB simulation (folder AMR ROS)

File	What it does	Output
simulate_industry_amr_warehouse.m	Script simulation: 26 x 16 m warehouse with racks, pallets, a ramp to a loading dock, the Scout Mini STL meshes, LiDAR, IMU and odometry drift, ISO 3691-4 style warning / braking / stop fields and automatic re-routing.	An animated 3D figure with an occupancy grid, speed / wheel-speed / slip plots and safety telemetry. Not a multibody simulation.
plan_warehouse_roadmap.m	Waypoint roadmap along aisle centre lines, avoiding hazard boxes.	path: list of waypoints from the current position to the goal.
teleop_industry_amr_warehouse.m	Same scene driven by hand (W/A/S/D, arrows, space).	Interactive 3D figure.

4. How the files work together

The call tree (also in section 12.2 of the report). Read it from the top: each indented file is called by the file above it.

```
run_scout_warehouse_simulation
  scout_amr_params ........ all constants (every file reads this)
  build_scout_amr_model ... only if scout_amr_sim.slx is missing
      smimport(scout_mini.urdf + meshes) -> scout_import_raw.slx
      writes scout_amr_sim.slx (MATLAB Function blocks call the two files below)
  sim(scout_amr_sim) .......... every 5 ms:
      Controller block -> scout_amr_controller
            scout_wh_nav
```

```
                scout_wh_world ... obstacles and movers at time t
                scout_wh_lidar ... 72 ranges, 10 Hz
                scout_wh_plan .... A* path, every 0.5 s or when blocked
             scout_amr_motor_limit ... torque-speed limit of the motor (per wheel)
      Tire blocks (one per wheel) -> scout_amr_tires -> forces on the wheel
  saves results/scout_warehouse_results.mat, scout_amr_drive_post -> motor and battery, draws
Fig. 15 to 20 and 30

make_example_style_outputs reads the .mat -> trajectory_map.png, signals.png, MP4
```

In one sentence: the parameters file feeds everyone, the Simulink model calls the controller and the tyre model, the controller calls the navigation, the navigation calls the world, LiDAR and planner, and the run script ties model, parameters and plots together.

Order of events when you run run_scout_warehouse_simulation:

- The runner loads scout_amr_sim.slx (builds it with build_scout_amr_model.m if missing) and sets tc_on (1 = slip management, 0 = off).
- Simulink runs. Every 5 ms the Controller block calls scout_amr_controller.m. That calls scout_wh_nav.m, which calls scout_wh_world.m (obstacle geometry), scout_wh_lidar.m (ranges) and scout_wh_plan.m (A* path).
- The controller returns four wheel torques. Four Tyre blocks call scout_amr_tires.m to turn wheel states into ground forces. Simscape Multibody applies the torques and forces and moves the chassis, wheels and movers.
- The sensors feed pose, velocity and wheel speeds back to the controller. The logs are stored.
- When the mission state reaches 'done' the simulation stops. The runner converts the logs, saves the .mat file, draws the figures and prints the summary.
- scout_amr_params.m is read by all of them, so changing a number there changes the whole simulation.

5. Results to remember

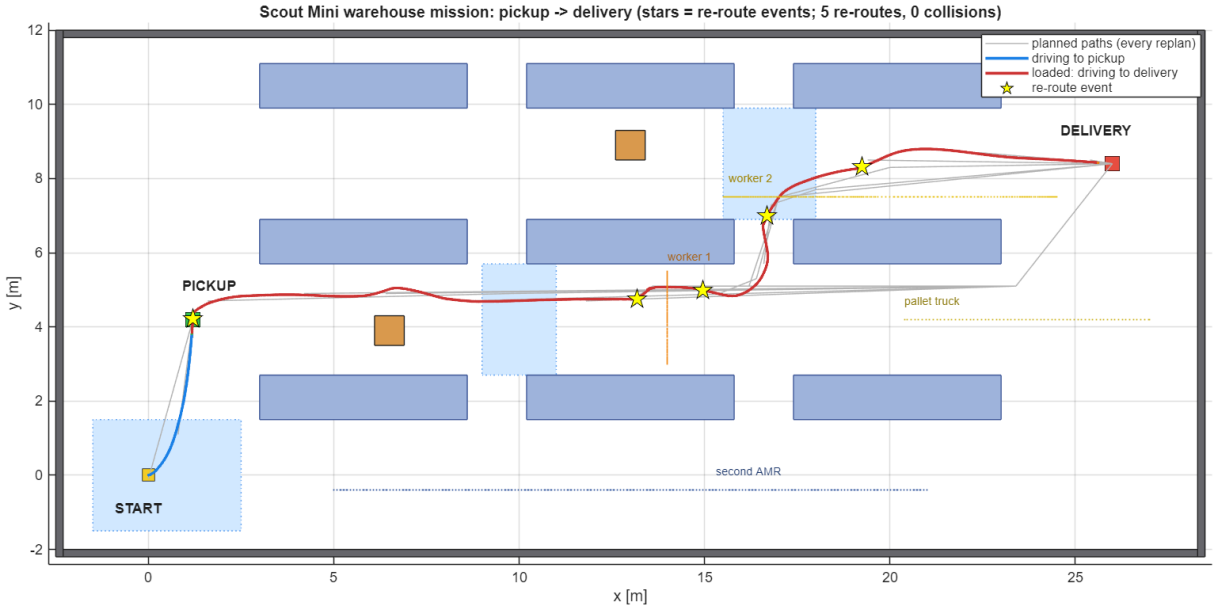
Quantity	Slip management ON	Slip management OFF
Mission time	51.5 s	45.5 s
Reached pickup / left / delivered	6.1 / 9.1 / 48.5 s	5.7 / 8.7 / 42.5 s
Distance	32.5 m	31.5 m
Top speed	1.20 m/s	1.19 m/s
Route corrections	5	3
Collisions	0	0
Minimum clearance	0.12 m	0.19 m
Peak slip ratio	0.46	0.63
Time with worst-wheel slip above 0.2	2.0 %	5.3 %
Wheel mechanical work	2717 J	2627 J
Peak motor current / peak wheel torque	9.2 A / 5.90 N·m	9.5 A / 6.06 N·m
Battery energy used / lowest voltage	1.06 Wh / 24.36 V	0.98 Wh / 24.37 V

Route corrections in the ON run happened at t = 9.7, 25.2, 32.2, 37.7, 41.7 s: the first as the robot leaves the pickup station (the LiDAR sees the floor box about 6 m ahead and the path is planned around it), the second and third when worker 1 comes within about 1 m in aisle 2, the fourth at the entry of aisle 3 and the fifth because of worker 2 in aisle 3. The pallet truck stops at x = 20.4 m and closes aisle 2, so the robot takes the cross-aisle at x of about 16.5 m. The ON and OFF runs are not identical: with slip management off the robot is faster and makes only 3 route corrections.

6. Every graph explained simply

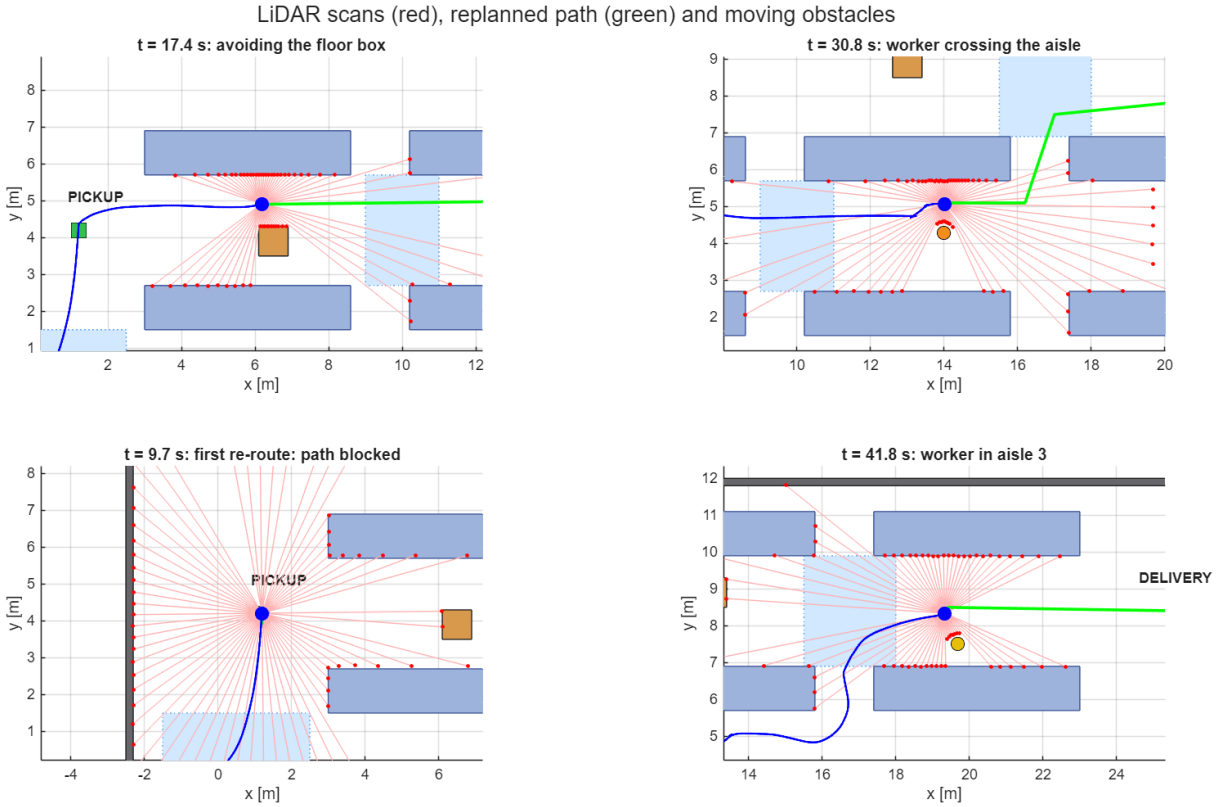
Each entry gives the picture, what it shows, how to read it and a sentence you can say. Coloured bands in the time graphs are the mission phases: blue = driving to the pickup, green = loading, orange = driving to the delivery, red = unloading. All numbers are from the reference run (slip management on).

Fig. 15 Warehouse map with the driven path



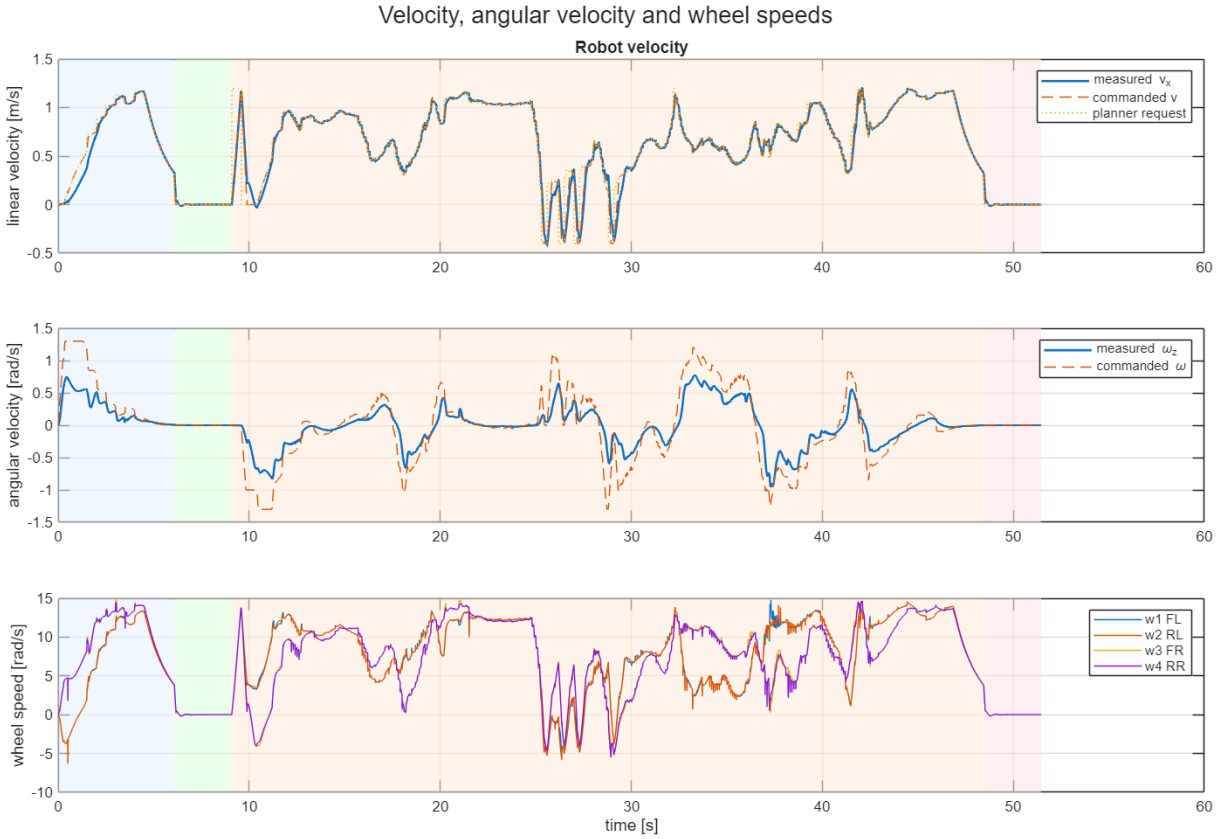
- **What it shows:** A top view of the whole warehouse. Blue boxes are the racks, light-blue areas are the oil and water patches, brown squares are floor boxes, and the dotted lines are the tracks of the workers, the second robot and the pallet truck.
- **How to read it:** Blue line = robot driving from START to PICKUP, red line = robot carrying the load from PICKUP to DELIVERY. Grey lines are the older plans that were replaced. The 5 yellow stars are the moments when the robot changed its route (the first one is at the pickup station).
- **Say this:** This is the whole mission on one picture: the robot goes up aisle 2, turns through the cross-aisle at about x = 16.5 m into aisle 3 and reaches the delivery point without touching anything.

Fig. 16 LiDAR snapshots at four moments



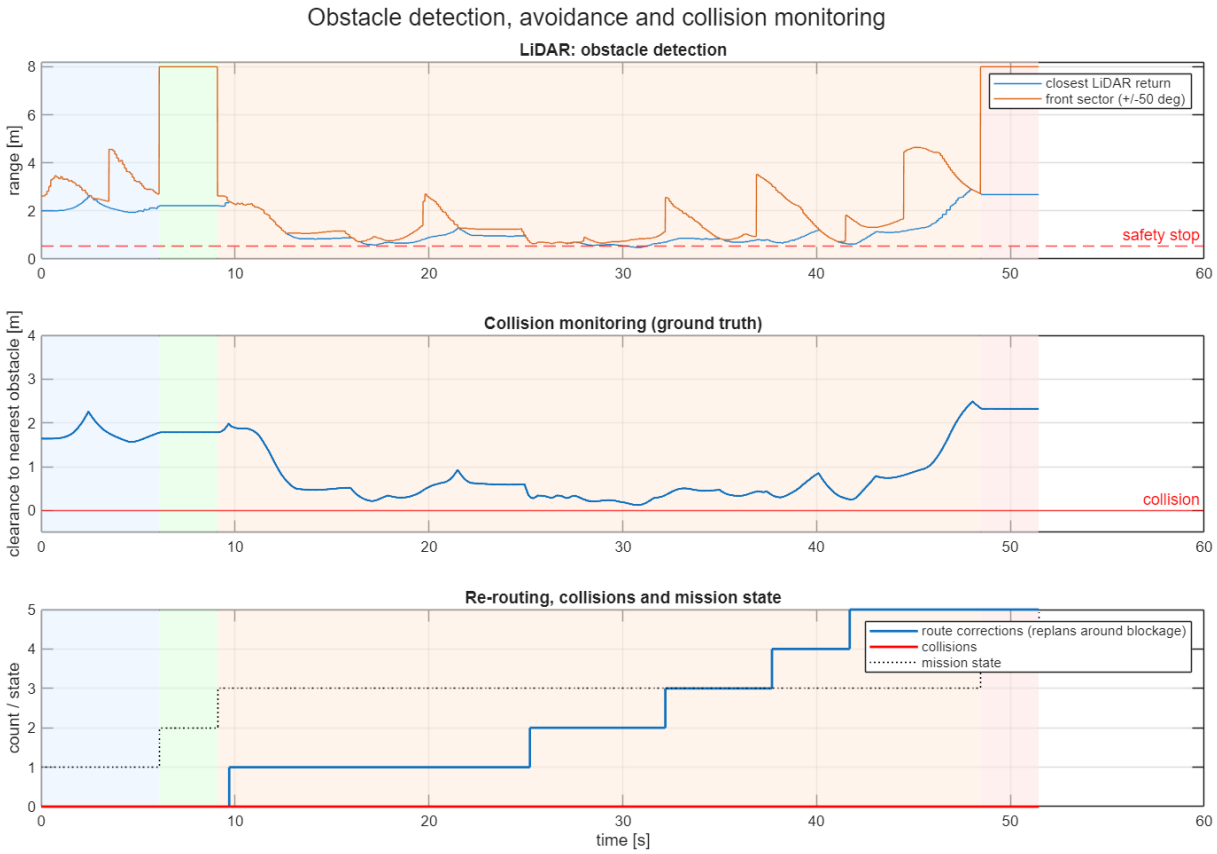
- **What it shows:** Four zoomed pictures of what the robot sees. Red lines are the LiDAR rays and the red dots are where they hit something. Blue line = path driven so far, green line = the current plan, orange and yellow circles = workers.
- **How to read it:** Bottom left (9.7 s): the first route correction at the pickup station; the LiDAR has just seen the floor box in aisle 2. Top left (17.4 s): the robot passes the floor box and the rays bunch up on it. Top right (30.8 s): worker 1 crosses aisle 2 right next to the robot (the smallest clearance of the run) and the plan leaves aisle 2 through the cross-aisle. Bottom right (41.8 s): worker 2 in aisle 3.
- **Say this:** The robot does not know about boxes, workers or the truck in advance. It only learns them from the LiDAR and plans again.

Fig. 17 Speeds and wheel speeds



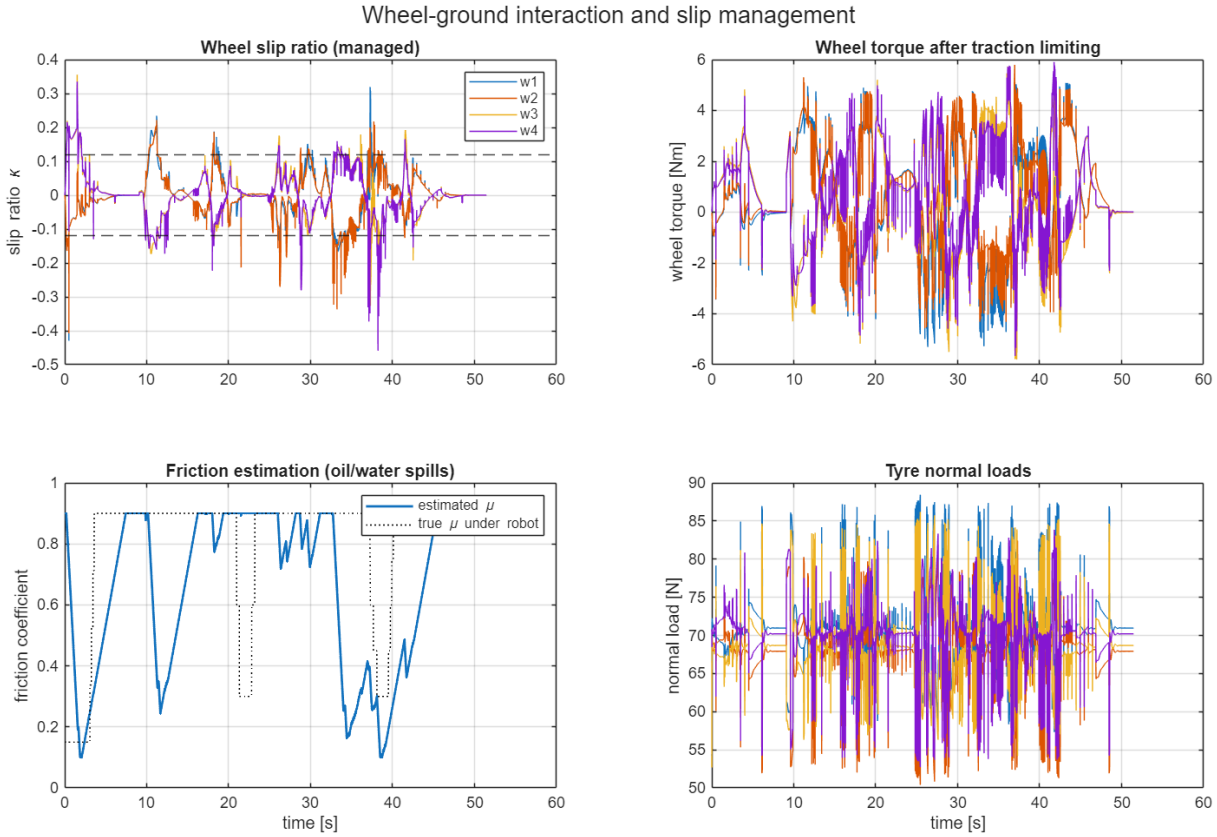
- **What it shows:** Top: forward speed. Middle: turning speed (yaw rate). Bottom: the speed of the four wheels. Commanded and measured curves are drawn together.
- **How to read it:** The measured line follows the commanded line closely, so the controller works. Top speed is 1.20 m/s. The dips below zero at about 25 to 30 s are the robot backing away from worker 1 and the pallet truck. In the bottom graph the left and right wheels differ whenever the robot turns, because a skid-steer robot steers by running its sides at different speeds. The highest wheel speed is 14.9 rad/s, below the motor knee at 15.5 rad/s.
- **Say this:** The robot accelerates smoothly, stops for 3 s at the pickup and the delivery, and turns by running its left and right wheels at different speeds.

Fig. 18 Obstacle detection and safety



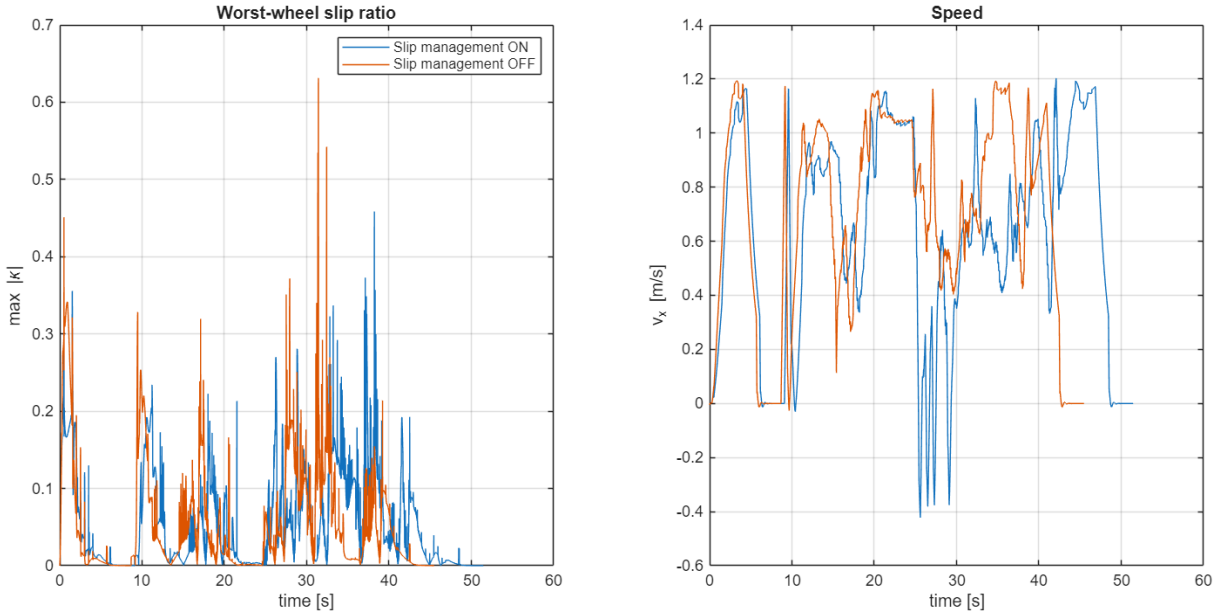
- **What it shows:** Top: distance to the nearest LiDAR return (blue) and to the nearest return in front (orange), with the 0.55 m safety-stop line. Middle: true clearance between the robot and the nearest obstacle. Bottom: number of route corrections (blue steps), collisions (red) and the mission state (dotted).
- **How to read it:** The clearance curve never reaches the red line at zero, so no collision happened. Its lowest value is 0.12 m (at 30.8 s, passing worker 1). The nearest return is 0.48 m, from the side; in front the nearest return never fell below 0.64 m, above the 0.55 m stop line. The blue steps at 9.7, 25.2, 32.2, 37.7, 41.7 s are the route corrections.
- **Say this:** Zero collisions, closest approach 12 cm. This is a small margin: worker 1 is a scripted obstacle that waits while the robot is close. The clearance is measured from the true geometry; the controller never sees it.

Fig. 19 Wheel slip and traction management



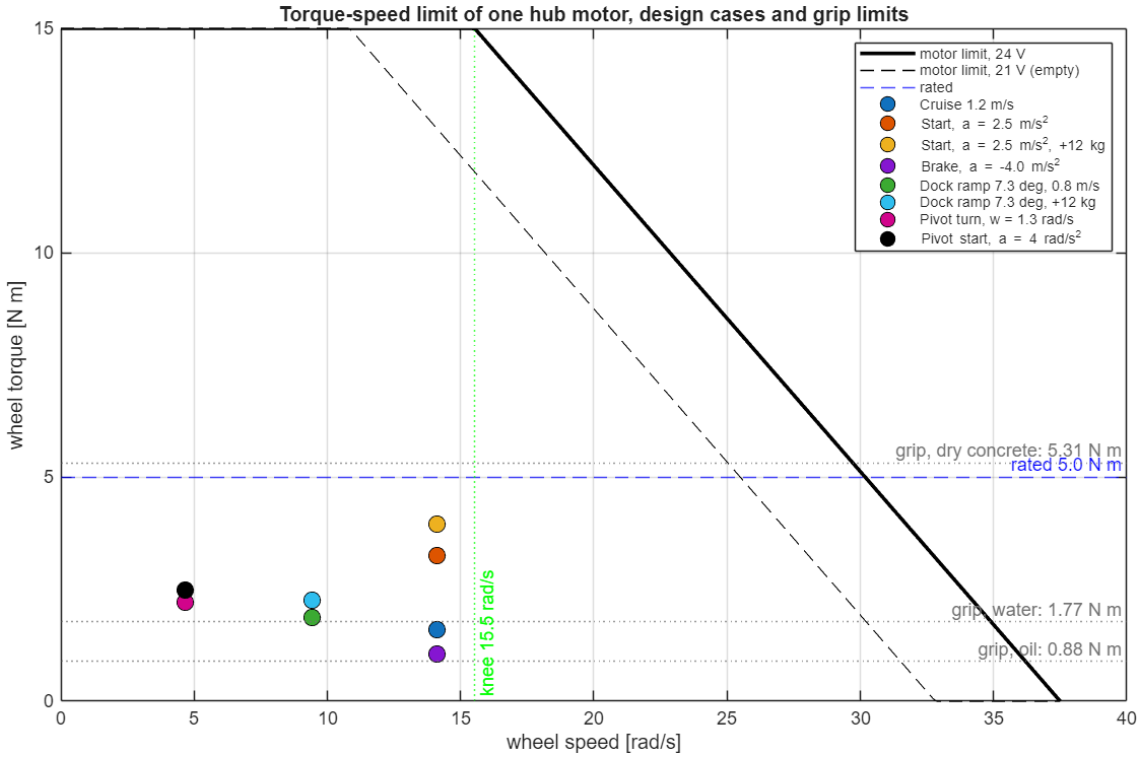
- **What it shows:** Four panels. Top left: slip ratio of each wheel with the plus and minus 0.12 detection lines. Top right: wheel torques after limiting. Bottom left: the friction the controller estimates (blue) against the true friction under the robot (dotted). Bottom right: the load on each tyre.
- **How to read it:** Slip stays mostly inside the 0.12 lines and spikes in sharp turns, in reversing and on the spills (peak 0.46). In the friction panel the true friction (dotted) drops to 0.15 on the oil at the start and to 0.30 on the water spills near 21 to 23 s and 37 to 40 s. The estimate falls when slip is detected and climbs back again. Wheel torque stays below 5.9 N·m, far below the 15 N·m motor peak.
- **Say this:** The controller notices slip, lowers its friction estimate and gives less torque and acceleration on a slippery floor. The noisy torque and load curves come from the stiff contact model and were not tuned away.

Fig. 20 Slip management on and off



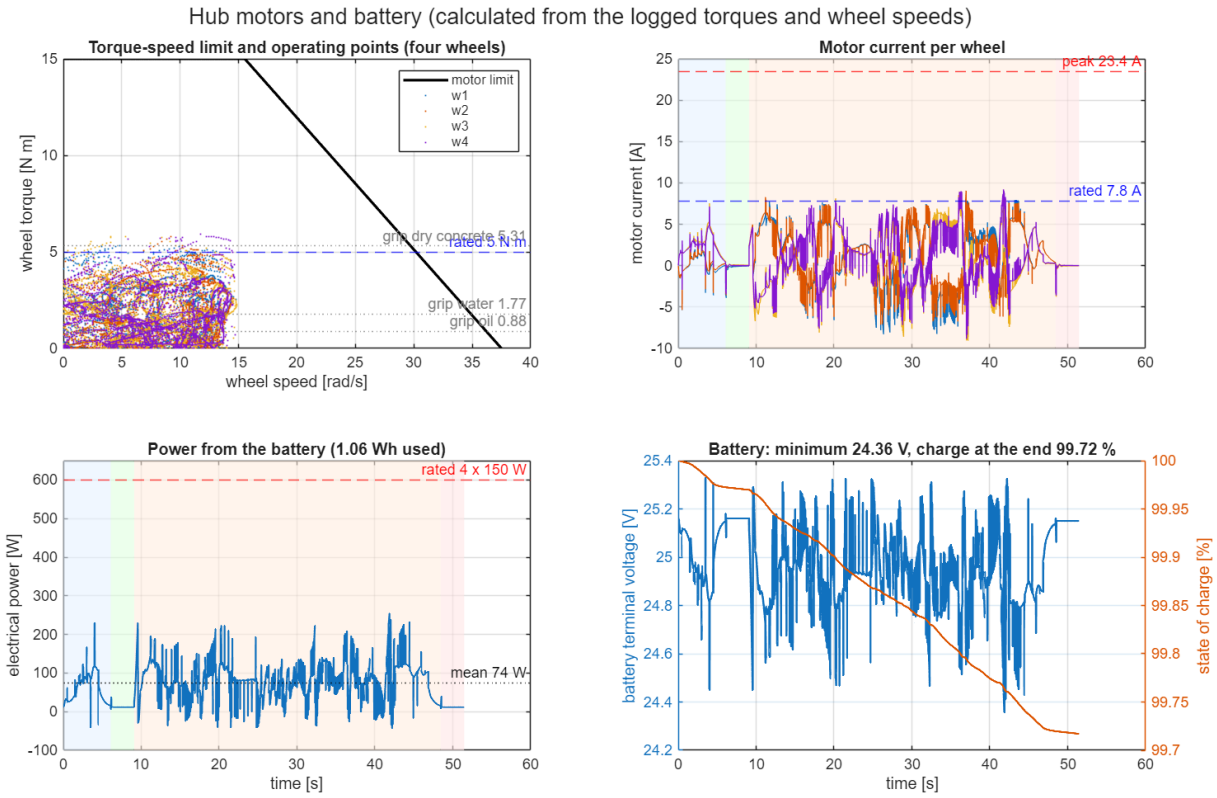
- **What it shows:** Left: the worst wheel's slip ratio for both runs. Right: forward speed. Blue = slip management on, orange = off. Same world, same mission, only the switch tc_on is different.
- **How to read it:** Orange (off) has the higher spikes: peak slip 0.63 against 0.46 for blue, and 5.3 % of the time above 0.2 against 2.0 %. Blue is slower: 51.5 s against 45.5 s.
- **Say this:** Slip management lowers the worst slip and makes the wheels behave better, and it costs about 6 s of mission time. Both runs finished without a collision, so it is not the difference between success and failure. The two runs also differ in their route corrections (5 and 3), so the comparison is one run per case.

Fig. 29 Torque-speed limit and the design cases (hand calculation)



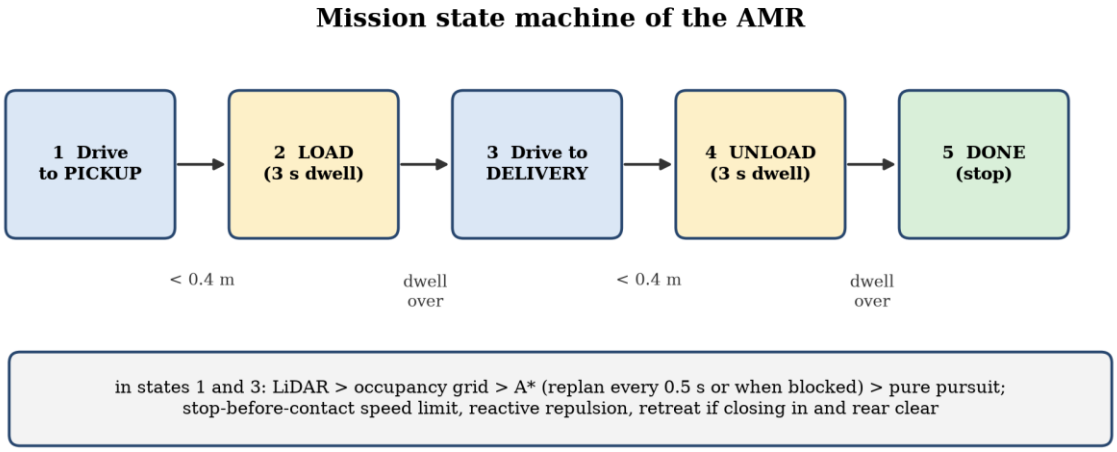
- **What it shows:** The torque that one hub motor can give against its wheel speed (thick line: full battery 24 V; dashed: empty battery 21 V), the eight driving cases that were calculated by hand (dots) and the grip limits of the floor (dotted horizontal lines).
- **How to read it:** The motor gives its peak torque of 15 N·m up to the knee at 15.5 rad/s and less above it, down to zero at 37.5 rad/s. All eight dots lie far below the line, so the motor is large enough: the highest case (start with 12 kg load) needs 3.95 N·m, which is 79 % of the rated 5 N·m. The grip limits are lower than the motor limit: the floor, not the motor, limits the torque that can be used (5.31 N·m on dry concrete, 1.77 N·m on water, 0.88 N·m on oil).
- **Say this:** The motor is not the weak point; the tyre grip is. That is why the slip management is needed on the oil and water patches.

Fig. 30 Motor and battery during the mission



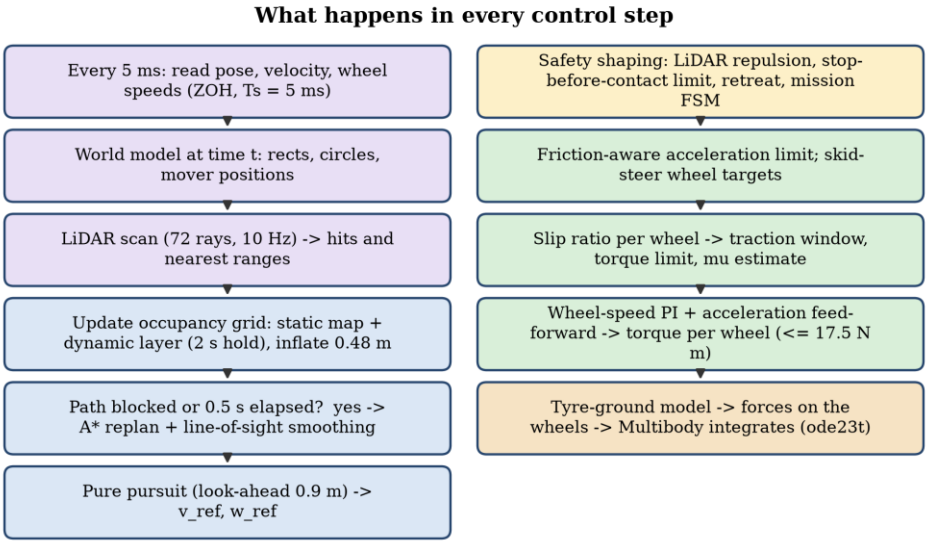
- **What it shows:** Four panels, calculated from the logged wheel torques and speeds after the run. Top left: all operating points of the four wheels against the motor limit. Top right: motor current. Bottom left: electrical power from the battery. Bottom right: battery terminal voltage (blue) and state of charge (orange).
- **How to read it:** The points stay under the line, so the limit is never reached. The motor current reaches 9.2 A, a little above the rated 7.8 A but far below the 23.4 A peak; only 0.5 % of the samples are above the rated torque. The mean power is 74 W (peak 254 W, with a short negative part while braking). The mission uses 1.06 Wh, the voltage never falls below 24.36 V and the charge ends at 99.7 %.
- **Say this:** The 24 V, 15 Ah battery is far larger than this single 51 s mission needs. These values are calculated after the run and do not act back on the dynamics. The motor constants, the winding resistance and the battery resistance are assumptions, so they are not measured values.

Fig. 14 Mission state machine



- **What it shows:** The five steps of the job: drive to pickup, load, drive to delivery, unload, done.
- **How to read it:** A drive step ends when the robot is within 0.4 m of the goal. A load or unload step ends after 3 s.
- **Say this:** The mission logic is a simple state machine. Inside the drive steps the robot senses, plans and steers.

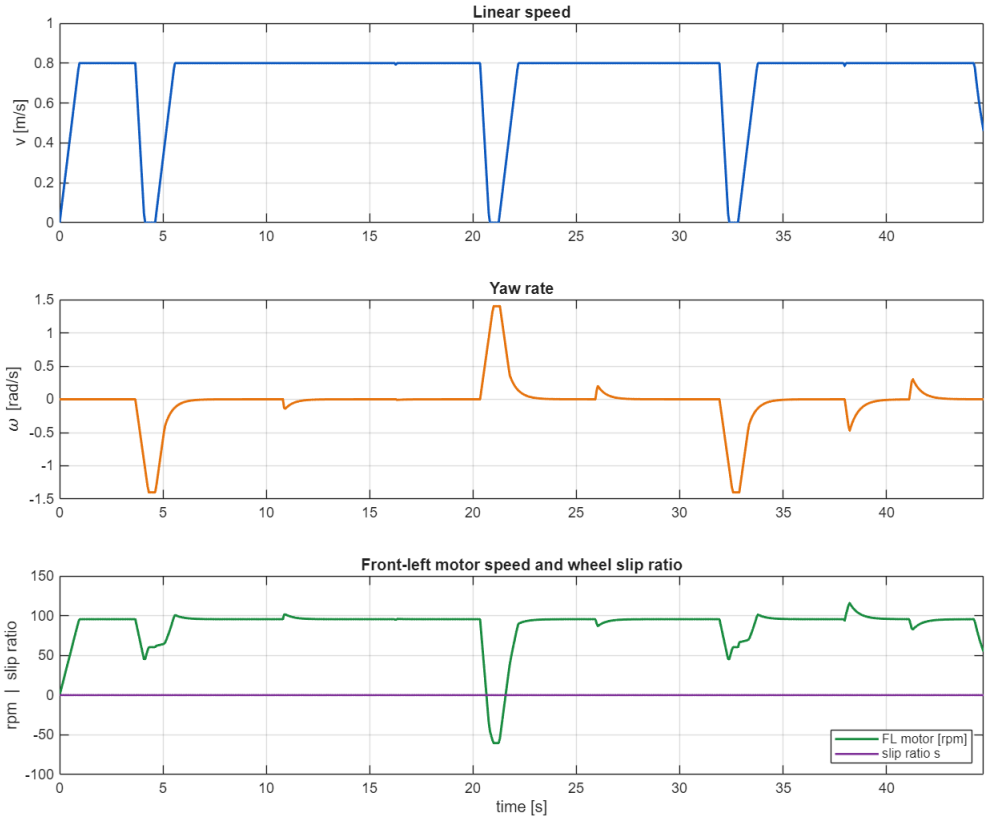
Fig. 22 One control step (every 5 ms)



- **What it shows:** What the controller does 200 times per second, from reading the sensors to giving the wheel torques.
- **How to read it:** Read the boxes from the top of the left column down, then the right column. Purple = sensing, blue = planning, yellow = safety rules, green = wheel control, orange = physics.
- **Say this:** Perception, planning, safety shaping, slip control and torque all happen inside every 5 ms step. A* is only run again every 0.5 s or when the path is blocked.

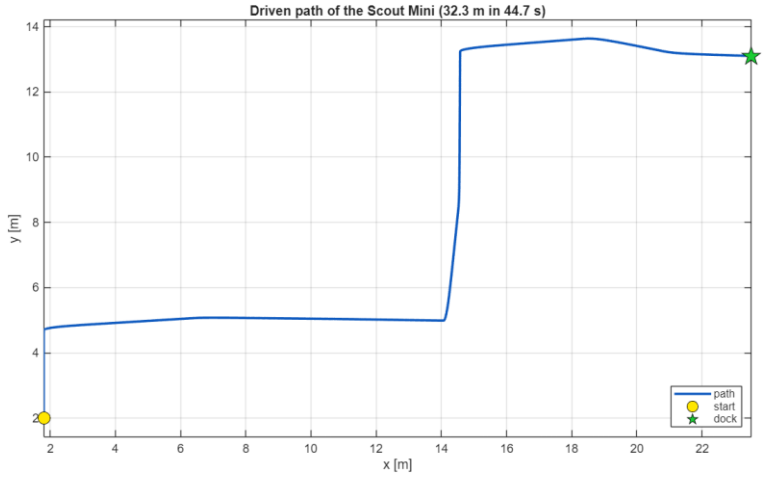
Graphs of the earlier script simulation (simulate_industry_amr_warehouse.m, a kinematic model, not a multibody simulation):

Fig. 24 Speed, yaw rate, motor speed and slip (script)



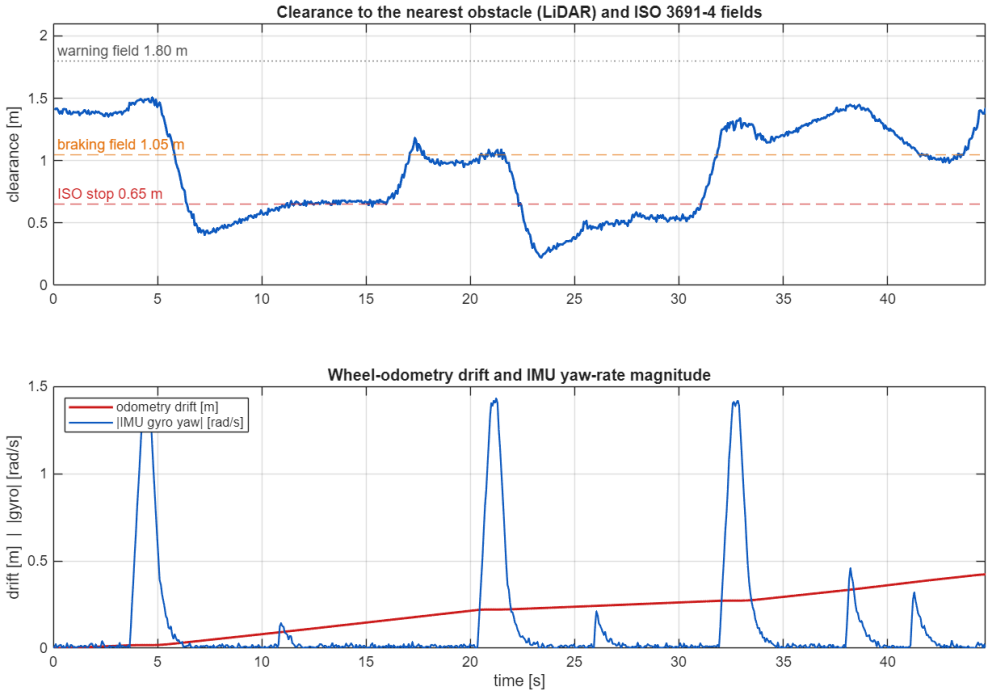
- **What it shows:** Linear speed (limit 0.8 m/s), yaw rate (limit 1.4 rad/s), front-left motor speed and the slip ratio of the script robot.
- **How to read it:** The speed drops to zero at the three corners of the route, where the robot pivots on the spot. The slip ratio is flat at zero.
- **Say this:** The slip is zero here only because this script has a simple drift model and no tyre contact. The Simscape model of Fig. 19 is the one that really simulates slip.

Fig. 25 Path driven by the script



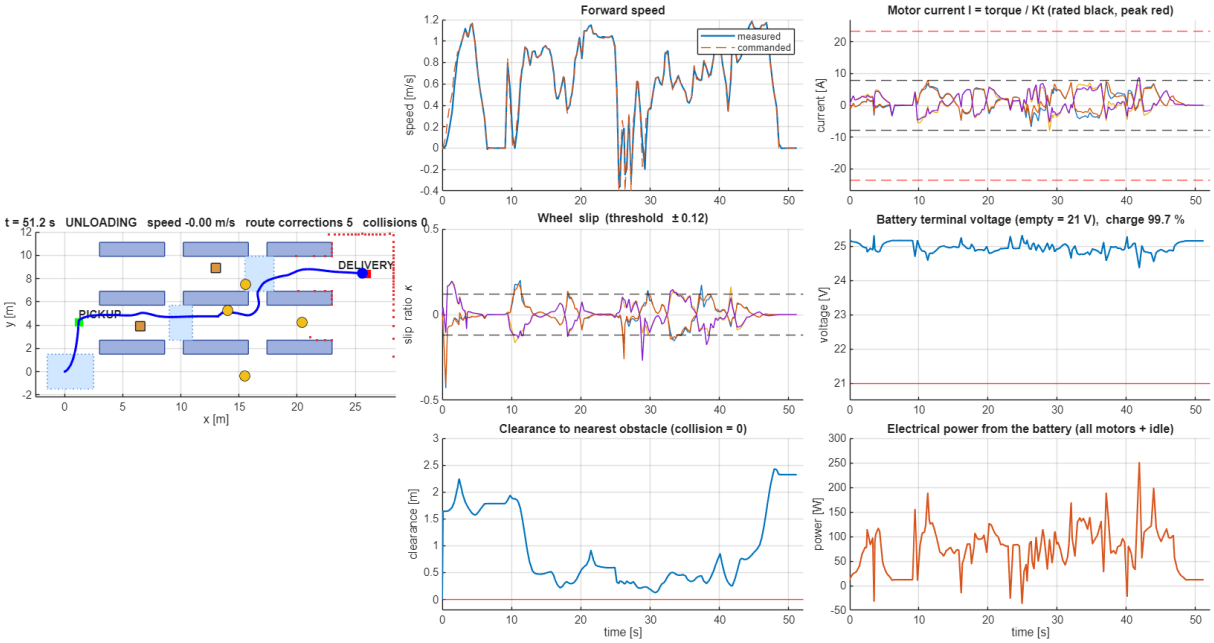
- **What it shows:** The route from the inbound depot to the shipping dock.
- **How to read it:** It follows the aisle centre lines of the waypoint roadmap and turns at three corners.
- **Say this:** The route comes from plan_warehouse_roadmap.m: a fixed roadmap along the aisles, not a grid search like the A* of the main model.

Fig. 26 Clearance with the ISO 3691-4 fields, drift and IMU (script)



- **What it shows:** Top: LiDAR clearance with the warning (1.80 m), braking (1.05 m) and protective-stop (0.65 m) fields. Bottom: wheel-odometry drift (red) and the IMU yaw-rate magnitude (blue).
- **How to read it:** The clearance dips below the braking field between the racks and has its lowest value of 0.22 m at about 23 s. The three tall blue peaks are the corner turns. The red line shows odometry drift growing to 0.42 m.
- **Say this:** This shows how safety fields slow the robot down, and why wheel odometry alone drifts and a real robot needs LiDAR-based localisation.

Fig. 27 Live figure while the simulation runs



- **What it shows:** A screenshot of live_graphs at the end of the mission (t = 51.4 s). The map on the left shows the robot, its driven path (blue), the planned path (green), the LiDAR hits (red dots) and the moving obstacles. In the middle: forward speed, wheel slip and clearance. On the right: motor current, battery voltage (the title shows the charge) and electrical power.

- **How to read it:** Everything updates about every 0.1 s while the robot drives. The current lines have dashed limits at the rated 7.8 A (black) and the peak 23.4 A (red). The voltage stays far above the 21 V empty line. The power plot starts at the idle power of 12 W and peaks when the robot accelerates.
- **Say this:** Command: live_graphs. This is the same data as the other graphs, but seen live. The report numbers come from run_scout_warehouse_simulation.

7. Likely questions and short answers

Q: Why Simscape Multibody and not just a MATLAB animation?

A: Because the wheels, chassis and tyre forces are simulated with real physics: torque on the wheels, friction at the ground, mass and inertia. The motion (and the slip) comes out of the physics, not from a drawn path.

Q: How did the SolidWorks model get into MATLAB?

A: SolidWorks to URDF + STL meshes with the exporter, then importrobot (Robotics System Toolbox) and smimport (Simscape Multibody) read the URDF.

Q: What changed after the import?

A: The weld between the footprint and the chassis was replaced by a 6-DOF joint so the robot can move; the wheel joints were set to accept torque; sensors, tyre forces, the warehouse and the controller were added.

Q: How is wheel-ground friction modelled?

A: Per wheel: a spring-damper normal force and a smoothed Coulomb friction force with μ from a floor map (0.9 concrete, 0.15 oil, 0.30 water). Written in scout_amr_tires.m.

Q: What is slip ratio and how is it handled?

A: Slip ratio = (wheel speed x radius - ground speed) / reference speed. Above 0.12 the controller limits the wheel-speed target to the ground speed window, cuts torque as slip grows and lowers its friction estimate, which also lowers the acceleration limit.

Q: Did slip management help?

A: It reduced the peak slip (0.46 vs 0.63) and the time with high slip, but made the mission about 6.0 s slower. The robot also finished without collision with it off, so the benefit is smoother wheel behaviour, not success versus failure.

Q: How does the robot see obstacles?

A: A 72-beam, 8 m, 10 Hz LiDAR modelled by ray casting against the warehouse geometry. Racks and walls are known in advance; boxes, workers, the second AMR and the truck appear only when the LiDAR hits them.

Q: How does it plan and avoid?

A: A* on a 0.2 m occupancy grid inflated by 0.48 m, re-planned every 0.5 s or when the path is blocked, then pure pursuit with reactive repulsion and a stop-before-contact speed limit. If an obstacle closes in and the rear is free it backs away.

Q: What does the mission state machine do?

A: Five states: drive to pickup, load 3 s, drive to delivery, unload 3 s, done.

Q: Is the pose estimated?

A: No. The pose is the true simulator pose from the Transform Sensor blocks. There is no localisation or SLAM in the Simulink model.

Q: Are collisions physical?

A: No. Collisions are checked analytically with a 0.36 m robot circle against the geometry. There is no Simscape contact between the robot and obstacles, and the movers follow scripted paths.

Q: Is the IMU or camera used?

A: They are mounted on the robot (from the URDF) but give no signals in Simulink. Their specifications are listed in the report as part of the robot description; the IMU, odometry drift and LiDAR noise are modelled only in the earlier script simulation.

Q: How is the motor modelled?

A: A hub motor with a torque constant of 0.64 N·m/A, a winding resistance of 0.6 ohm, 24 V supply, 5 N·m rated and 15 N·m peak torque. The torque limit is min(15, Kt (24 - Ke w) / R): flat up to the knee at 15.5 rad/s and falling to zero at 37.5 rad/s. The controller obeys this limit. The values are assumed, not measured.

Q: How was the torque calculated?

A: The force at the wheels is the rolling resistance plus the slope plus the mass times the acceleration; the wheel torque is the force times the wheel radius (0.085 m) divided by four wheels, plus the inertia and the joint damping. The cruise at 1.2 m/s needs 1.59 N·m per wheel; the start at 2.5 m/s² with 12 kg load needs 3.95 N·m (79 % of rated). The simulation gives the same torques within a few percent.

Q: Why does the motor current not change the robot movement?

A: The motors are modelled as torque sources with the torque-speed limit. The current, power and battery are calculated after the run from the same torques and speeds. A full electrical model (Simscape Electrical) is listed as future work.

Q: Is the battery big enough?

A: Yes: the mission uses 1.06 Wh of the 360 Wh (24 V, 15 Ah), the voltage stays above 24.4 V and the peak battery current is 10.4 A.

Q: What limits the torque the robot can use?

A: The grip of the floor: mu times weight times wheel radius divided by four is 5.31 N·m on dry concrete, 1.77 N·m on water and 0.88 N·m on oil. That is below the 15 N·m motor limit, so the slip management reduces the torque on slippery floor.

Q: How do I change the warehouse or the gains?

A: Edit scout_amr_params.m. If you change the racks, walls or movers, rebuild the model with build_scout_amr_model.m so the 3D scene matches.

8. Honest limits (say these before you are asked)

- Simulation only. Nothing was tested on the physical Scout Mini.
- LiDAR and collision detection are analytic; there is no physical contact with obstacles.
- The moving obstacles follow scripts, and the workers pause when the robot is close.
- The pose is the true simulator pose (no localisation, no sensor noise in Simulink).
- The tyre model is a simple point-contact model; the motors are torque sources limited by the torque-speed envelope (15 N·m peak); the motor current and the battery are calculated after the run and do not act on the dynamics, and the motor and battery data are assumed.
- The 3D scene of the Simscape model is shown by the Multibody Explorer screenshots (Fig. 11-13); the live view appears when the model runs.

- The slip comparison is one run per case on one layout, and the result is sensitive to small parameter changes (the number of route corrections changed from 3 to 5 when the motor limit was added).
- The smallest clearance is only 0.12 m, from a scripted worker that waits while the robot is close; the collision circle (r = 0.36 m) is a simplification of the chassis outline.
- The slip ratio uses a track half-width of 0.30 m while the model has 0.306 m (about 2 % in the slip calculation).

PART 2: LIVE DEMO CHEAT SHEET

Design and Simulation of Industry Standard AMRs (AgileX Scout Mini) - MATLAB R2025b

0. Before the demo

Type these once in the MATLAB Command Window. Every later command assumes this folder. A ready-made script, live_demo.m, holds all commands below in sections: open it in MATLAB and run one section at a time with Ctrl+Enter.

Step	What it does	Command
1	Go to the project folder and add the root folder (script simulation) to the path	`cd 'D:\academic_shit\SEM5-III\AMR ROS\simulink_amr'` `addpath(pwd); addpath('..');`
2	Close old copies of the model and load the parameters (do this if anything looks wrong)	`close_system('scout_amr_sim',0); bdclose all;` `scout_amr_params;`
3	Open live_demo.m (all commands, one section per figure)	`edit live_demo`

1. Command for each figure of the report

Report item	What it shows	Command
Fig. 2-4 SolidWorks model	The CAD assembly (needs SolidWorks); otherwise show the pictures in the report	`winopen('..\scout_mini_sw\Assem2.SLDASM')`

Fig. 6 importrobot	URDF read by MATLAB as a rigidBodyTree, rotate with the mouse	`rbt = importrobot('scout_mini.urdf');`
		`show(rbt, homeConfiguration(rbt),`
		`'Visuals', 'on');`
		`showdetails(rbt)`

Fig. 7 CAD meshes + wheel axes	Meshes placed with the URDF transforms (saved picture)	`imshow('report/figs/robot_render.png')`
Fig. 9 Simulink top level	World, Mechanism Configuration, Solver and the four colour-coded subsystems	`open_system('scout_amr_sim')`
Fig. 10 Mission_and_Slip_Controller	Zero-order holds, Controller block, logging; tc_on switch	`open_system('scout_amr_sim/Mission_and_Slip_Controller')`

Section 7 subsystems	Robot, tyre model (one MATLAB Function per wheel), warehouse, movers	`open_system('scout_amr_sim/Scout_Mini_Robot')`
		`open_system('scout_amr_sim/Scout_Mini_Robot/Tyre_Ground_Contact_Model')`
		`open_system('scout_amr_sim/Warehouse')`
		`open_system('scout_amr_sim/Moving_Obstacles')`

Fig. 11-13 Multibody Explorer	The 3D Simscape scene; the robot drives the mission live (about 1.5 minutes). Buttons Iso / Top change the view	`scout_amr_params;`
		`open_system('scout_amr_sim');`
		`set_param('scout_amr_sim','SimMechanicsOpenEditorOnUpdate','on');`
		`set_param('scout_amr_sim','SimulationCommand','update');`
		`set_param('scout_amr_sim','SimulationCommand','start');`

Fig. 15-18, 19 Result graphs	Trajectory, LiDAR scenarios, velocities, safety, slip: redrawn from the saved run (a few seconds)	`run_scout_warehouse_simulation('replot')`
Fig. 20 Slip ON vs OFF	Comparison figure from the saved run	`imshow('results/6_slip_on_vs_off.png')`

Fig. 1, 5, 8, 14, 21, 22 Diagrams	Toolchain, URDF tree, robot subsystem, mission state machine, architecture, control step	`imshow('report/figs/diag_architecture.png')`
		`% also diag_toolchain, diag_urdf_tree,`
		`diag_robot_subsystem, diag_mission_fsm,`
		`diag_control_step`

Trajectory map (not in report)	Top-down map with footprints and LiDAR fans	`make_example_style_outputs(false);` `imshow('results/trajectory_map.png')`

Animation (GIF / MP4)	Top-down animation of the whole mission	`winopen('results/amr_warehouse_avoidance.mp4')`
		`% rebuild (a few minutes):`
		`make_example_style_outputs`

Fig. 27 Live graphs	One figure that updates while the robot drives: map, speed, slip, clearance, motor current, battery voltage and electrical power. Add (false) for no 3D window	`live_graphs` `live_graphs(false)`
Fig. 28-29 Motor, torque and grip (hand calculation)	Table of the 8 driving cases (force, torque, current, power, % of rated), the grip limits and the motor limits; then the torque-speed figure	`scout_amr_drive_sizing` `imshow('results/8_drive_design_cases.png')`
Fig. 30 Motor and battery of the mission	Torque-speed limit with the operating points, motor current, power, battery voltage and charge (redrawn from the saved run)	`run_scout_warehouse_simulation('replot')` `imshow('results/7_motor_battery.png')`
Unit tests of the drive model	Six tests: stall torque, no-load speed, knee, hand values, grip, battery	`runtests('test_scout_amr_drive')`
Fig. 23 Script simulation	ISO 3691-4 style simulation: camera buttons, Replan Route, Emergency Stop (runs about 45 s)	`cd('..'); simulate_industry_amr_warehouse;` `cd('simulink_amr');`
Manual driving (optional)	Same robot with the keyboard: W/A/S/D or arrows, Space = stop	`cd('..'); teleop_industry_amr_warehouse;` `cd('simulink_amr');`
Fig. 24-27 Script telemetry	Speed, yaw rate, slip, clearance with ISO fields, drift, path	`addpath('report'); make_industry_graphs;` `imshow('report/figs/industry_g2_safety_sensors.png')`

2. Running the full simulation live

Goal	Command	Note
Mission with 3D Multibody Explorer, then all graphs	run_scout_warehouse_simulation	about 1.5 minutes; replaces results/ with the slip-management-ON run only
Faster, no 3D window	run_scout_warehouse_simulation(false)	headless, about 1.5 minutes
Slip management ON and OFF with comparison (Fig. 20)	run_scout_warehouse_simulation(false, true)	about 2 minutes; keeps both runs
Slip management off by hand	set_param('scout_amr_sim/Mission_and_Slip_Controller/tc_on','Value','0') sim('scout_amr_sim')	set it back to '1' afterwards
Rebuild the model from the URDF	build_scout_amr_model	reuses scout_import_raw.slx; the first import takes several minutes
Open the finished report	winopen('..\AMR_report_documents\Scout_Mini_Warehouse_AMR_Report_Motor_Torque.pdf')	full report, 50 pages
Open the second-review report	winopen('..\AMR_report_documents\Scout_Mini_Warehouse_AMR_Second_Review.pdf')	25 pages

3. Suggested order for a 10-minute live demo

1. Setup (step 0), then open_system('scout_amr_sim'): explain the four subsystems (Fig. 9), open the controller (Fig. 10) and the tyre model.
2. Run the Multibody Explorer lines (Fig. 11-13) or live_graphs: the robot drives the warehouse mission while you describe LiDAR, A* re-planning and the moving obstacles; live_graphs also shows the motor current and the battery.

3. run_scout_warehouse_simulation('replot'): walk through the trajectory, safety and slip graphs (Fig. 15-18, 19).
4. Motor and torque: run scout_amr_drive_sizing, then show Fig. 29 and Fig. 30: the motor limit is 15 N·m, the mission needs at most 5.9 N·m, the grip of the floor (5.3, 1.8 and 0.9 N·m) is the real limit, the battery uses 1.06 Wh.
5. Show Fig. 20 and state the honest result: slip management lowers the peak slip (0.46 against 0.63) and costs about 6 s of mission time; both runs are collision-free.
6. Show trajectory_map.png and the MP4 animation.
7. Finish with simulate_industry_amr_warehouse (the earlier script simulation, Fig. 23): use the camera buttons and Emergency Stop.

4. If something goes wrong

Problem	Fix
Multibody Explorer does not appear	Run the four Fig. 11-13 lines (the 'update' command opens it); check the taskbar, it may be behind other windows.
Undefined variable / parameter errors in the model	scout_amr_params (then run again)
Old model copy open or the model looks different	close_system('scout_amr_sim',0); bdclose all; open_system('scout_amr_sim')
Figures show the wrong run or OFF data is missing	run_scout_warehouse_simulation(false, true) (re-creates both runs)
simulate_industry_amr_warehouse cannot find plan_warehouse_roadmap	addpath('..') or cd('..') before running it
Window stays open after the demo	close all; bdclose all

5. Explanation of every graph: what it shows and why it is used

Part A - Simscape Multibody model (sections 7 to 12 and 17 of the report). All numbers are from the reference run with slip management on: 51.5 s, 32.5 m, 5 route corrections, 0 collisions, minimum clearance 0.12 m.

Figure	What is plotted	Why it is used	How to read it / what to say
Fig. 1, 14, 21, 22 Diagrams	Toolchain, mission state machine, closed-loop architecture, flow of one control step.	Explain the closed loop and the software structure without code.	Fig. 21 has four layers: Plant (Simscape) -> Perception (sensors, world, LiDAR) -> Autonomy (FSM, grid + A*, pure pursuit) -> Control (PI + slip management) -> wheel torques back to the Plant via the tyre model (F, M).
Fig. 2-4 SolidWorks views	Isometric, side and front view of the Scout Mini CAD (chassis, top cover, four wheels, axle brackets).	Shows that the robot is the author's own design and where the geometry comes from.	Say: the CAD cannot be opened in MATLAB, so it is exported to URDF and STL (section 5).
Fig. 5 URDF tree	Links and joints of scout_mini.urdf: base_footprint, base_link, four wheel joints, fixed sensor mounts.	Shows how the robot is described before it is imported.	Wheels are continuous joints; LiDAR, IMU and camera are fixed mounts; the fixed base_footprint joint is replaced by a 6-DOF joint in Simulink.
Fig. 6 importrobot	The robot as a rigidBodyTree in MATLAB.	Checks that the URDF and the STL meshes were read correctly (links, joints, wheel positions).	Rotate it live; four wheels and the base link must be at the right places.
Fig. 7 CAD meshes + wheel axes	Meshes placed with the URDF transforms; red arrows are the wheel joint axes.	Proves the wheel axis direction, which fixes the sign of wheel torque and slip ratio.	The imported joint frames have their Z axis along the axle; this was found here and built into the tyre model.
Fig. 8 Scout_Mini_Robot (schematic)	What the robot subsystem contains: 6-DOF chassis, wheel joints, sensors, wheel forces and the tyre model.	Explains the inside of the subsystem without the tangled physical wiring.	Torque goes in, wheel speed and pose come out, tyre forces are applied through External Force and Torque blocks.

Fig. 9 / 10 Simulink diagrams	Top level with four subsystems; inside of Mission_and_Slip_Controller.	Documents the model structure so that it is readable and repeatable.	Top level has no signal wires; signals travel by Goto/From tags; tc_on switches slip management.
Fig. 11-13 Multibody Explorer	Screenshots of the 3D Simscape scene at t = 0, 3.6 and 18.7 s.	Proof that the model runs as a real multibody simulation, not an animation.	Model tree: Moving_Obstacles, Scout_Mini_Robot, Warehouse, Mechanism Configuration, World.
Fig. 15 Warehouse map with trajectory	Top view of the hall; driven path (blue to the pickup, red to the delivery), all planned paths (grey), re-route events (stars), tracks of the moving obstacles (dotted).	The main picture of the mission: shows that the robot reached both stations through the aisles and where it re-planned.	5 stars = 5 route corrections (9.7, 25.2, 32.2, 37.7, 41.7 s); the first is at the pickup station. Pickup reached at 6.1 s, delivery at 48.5 s.
Fig. 16 LiDAR scenarios	Four snapshots: LiDAR scan (red), re-planned path (green), driven path (blue), moving obstacles.	Shows how the robot perceives and avoids: the floor box, the crossing worker, the blocked aisle, worker 2 in aisle 3.	Perception (red) -> new plan (green) -> motion (blue): the planner and the LiDAR work together.
Fig. 17 Velocities and wheel speeds	Linear speed and yaw rate (commanded and measured), and the four wheel speeds against time.	Shows that the controller follows its commands and that left and right wheel speeds differ in turns (skid steering).	Speed up to 1.2 m/s; the measured speed follows the command; flat parts are loading and unloading (3 s each).
Fig. 18 Perception and safety	LiDAR ranges, ground-truth clearance, route corrections and mission state (shaded bands).	Safety evidence: clearance to every obstacle, counted collisions, and when each mission state is active.	Clearance never reaches zero (minimum 0.12 m at 30.8 s, worker 1); 0 collisions; the closest LiDAR hit was 0.48 m (from the side), in front 0.64 m.
Fig. 19 Slip management	Wheel slip ratios with the +- 0.12 threshold, wheel torques after limiting, friction estimate against the true friction, tyre normal loads.	Core of the slip topic: shows how slip is detected, how the torque is limited and how the robot estimates the floor friction.	The estimate falls to 0.10 on oil and water and recovers on dry floor with a lag of seconds; normal load is about 70 N per wheel (28.3 kg / 4); torque stays below 6 N·m (motor peak 15 N·m); torque and load traces are noisy because the contact model is stiff.
Fig. 20 Slip ON vs OFF	Worst-wheel slip ratio and forward speed for both runs (blue ON, orange OFF).	The fair comparison: same world and mission, only tc_on changes.	ON: peak slip 0.46, 2.0 % of the time above 0.2, 51.5 s. OFF: peak slip 0.63, 5.3 %, 45.5 s. Both collision-free: less slip costs about 6 s. One run per case; the route corrections differ (5 and 3).
Fig. 29 Torque-speed limit and design cases	Motor limit (24 V solid, 21 V dashed), the 8 hand-calculated driving cases as dots, the grip limits as dotted lines.	Shows that the motor is large enough and that the floor grip is the real limit.	Knee at 15.5 rad/s, no-load speed 37.5 rad/s, stall 25.6 N·m (limited to 15). All 8 cases lie far below the line; the highest (start with 12 kg) is 3.95 N·m = 79 % of rated. Grip: 5.31 (dry), 1.77 (water), 0.88 N·m (oil).
Fig. 30 Motor and battery	Operating points of the four wheels against the motor limit, motor current, battery power, battery voltage and charge.	Shows what the mission asks from the motors and the battery.	Peak current 9.2 A (rated 7.8 A, peak 23.4 A); mean power 74 W; 1.06 Wh used; voltage never below 24.36 V. Calculated after the run, not feeding back into the dynamics.

Part B - MATLAB-script simulation simulate_industry_amr_warehouse.m (section 13). It is a kinematic and sensor-level model, not a multibody simulation: 44.7 s, 32.3 m, no collision reported.

Figure	What is plotted	Why it is used	How to read it / what to say
Fig. 23 Screenshots	The live window of the script (top view): warehouse and robot on the left; occupancy grid, kinematics and safety telemetry on the right.	`Shows the ISO 3691-4 style behaviour and the camera views (isometric, top, chase, dock).`	Occupancy grid (black = occupied) fills up as the LiDAR scans; the telemetry traces grow with time.
Fig. 24 Kinematics	Linear speed (limit 0.8 m/s), yaw rate (limit 1.4 rad/s), front-left motor speed, slip ratio.	`Shows the motion profile of the script robot.`	Speed drops to zero at the three route corners where the robot pivots; the slip ratio stays at zero because the script has only a drift model, not a tyre contact model.
Fig. 25 Driven path	x-y path from the inbound depot to the shipping dock.	`Shows the route that the waypoint roadmap produced.`	Up the west aisle, east along y = 5 m, north at x = 14.6 m, east to the dock: 32.3 m.
Fig. 26 Safety and sensors	LiDAR clearance with the 1.80 m warning, 1.05 m braking and 0.65 m stop fields; odometry drift; IMU yaw-rate magnitude.	`Shows how the safety fields are used and why wheel odometry alone is not enough.`	Clearance falls to 0.22 m between the racks; the 0.65 m stop field is not a hard limit here. Odometry drift grows to 0.42 m (2 % built-in drift); the IMU peaks are the three turns.

Honest limits to mention when asked: both simulations use the true robot pose (no localisation), analytic sensing and scripted moving obstacles; slip management gives smoother wheel-ground behaviour, not the difference between success and failure; the motor and battery data are assumed and the motor current does not act on the dynamics; the smallest clearance (0.12 m) depends on a scripted worker; nothing was tested on the real robot.